



Python for Fixed Income

COURSE 1 · PYTHON FOUNDATIONS

Introduction to Python

WEEK 1 · DAY 1

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

By the end of today you can

Store a bond

Its ticker, coupon, price, and what you hold.

```
coupon = 5.25
```

Ask a portfolio questions

Totals, averages, highest and lowest.

```
sum(face_list)
```

Repeat a calculation

Once for every bond, without retyping it.

```
for price in price_list:
```

Write your own function

Define it once. Use it anywhere.

```
def current_yield(coupon, price):
```

One desk, four bonds

Ticker	Issuer	Coupon	Price	Rating
QVNE	Quorvane Energy	5.25	98.50	BBB
DNMR	Dunmarrow Rail	6.125	103.25	BB+
PLWK	Pellwick Paper	4.75	95.00	A-
HLVF	Halvering Foods	7.50	101.00	B+

Every issuer in this course is invented.

How every notebook works

Q.

A question the desk would ask



Code

A few lines that answer it



Takeaway

What the output tells us

A variable is a named cell

Name on the left, value on the right

```
ticker = 'QVNE'  
coupon = 5.25  
maturity = 2033  
price = 98.50
```

From then on, use the name and Python fills in the value.

Four kinds of value

str	Text, always in quotes	'QVNE'
------------	------------------------	--------

float	A number with a decimal point	5.25
--------------	-------------------------------	------

int	A whole number	2033
------------	----------------	------

bool	True or False	True
-------------	---------------	------

Percent means divide by 100

Right

```
income = face * coupon / 100  
52500.0
```

Wrong, and Python raises no error

```
income = face * coupon  
5250000.0
```

Know roughly what the answer should be before you run it.

Mars Climate Orbiter, 1999

A ground software file supplied thruster data in English units. The trajectory models expected metric.

Both numbers were valid. The units did not match. Nothing raised an error.

The spacecraft arrived about 170 kilometers lower than planned.

Source: Mars Climate Orbiter Mishap Investigation Board, Phase I Report, November 10, 1999.

Three ways to hold more than one value

List

[]

One column of a sheet. Ordered, and it can be changed.

```
coupon_list =  
    [5.25, 6.125, 4.75, 7.5]
```

A column

Tuple

()

Like a list, but it cannot be changed after it is made.

```
frequencies = (1, 2, 4)
```

Something fixed

Dictionary

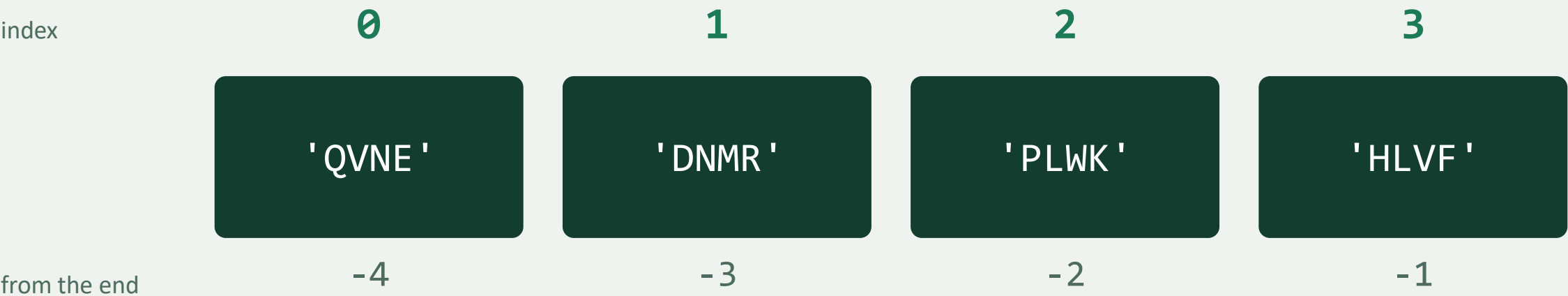
{ }

One row of a sheet. Each value has a name, called a key.

```
bond = {'Ticker': 'QVNE',  
        'Coupon': 5.25}
```

A record

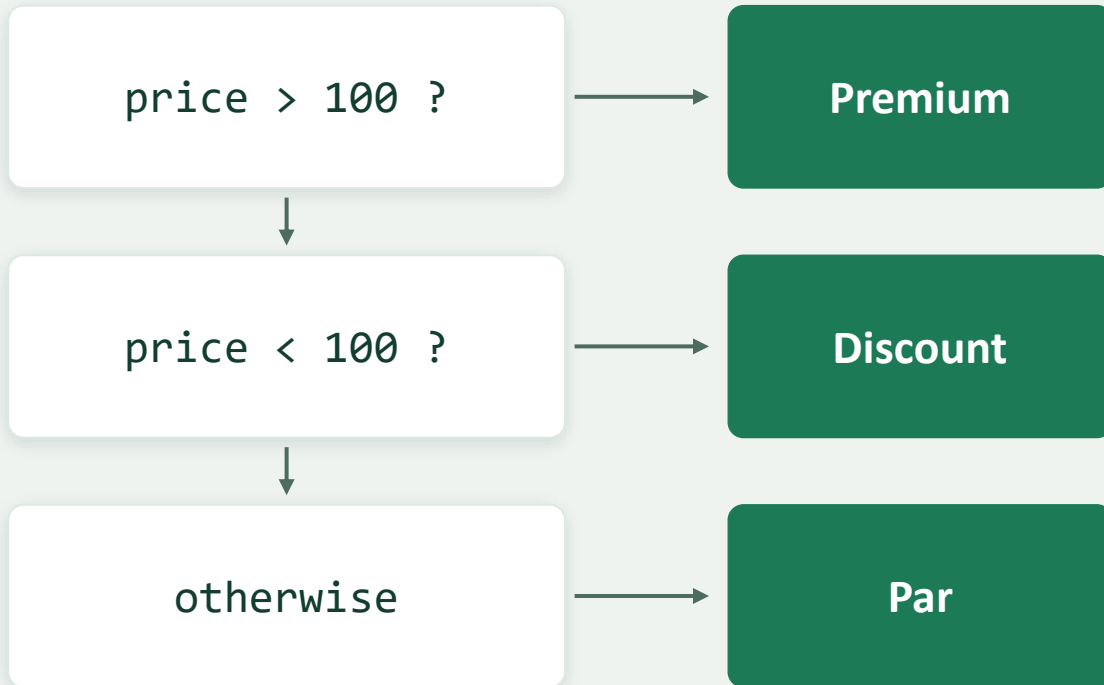
Python counts from zero



The first item is index 0. The last item is always index -1.

Python	<code>len(list)</code>	<code>sum(list)</code>	<code>min(list)</code>	<code>max(list)</code>
Excel	COUNT	SUM	MIN	MAX

Making a decision



```
if price > 100:  
    print('Premium')  
elif price < 100:  
    print('Discount')  
else:  
    print('Par')
```

Each test ends with a colon. The indented lines belong to it.

In Excel =IF(A1>100,"Premium",IF(A1<100,"Discount","Par"))

A loop is the fill-down

In Excel: one formula, dragged down the column

Face	Price	Cost
1,500,000	98.50	=A2*B2/100
500,000	103.25	=A3*B3/100
750,000	95.00	=A4*B4/100
250,000	101.00	=A5*B5/100

In Python: one formula, applied to every row

```
for face, price in zip(face_list,
                        price_list):
    cost = face * price / 100
    print(cost)
```

The running total

Start at zero, before the loop



Add to it, inside the loop



Read the answer, after the loop

The same loop in one line

As a loop

```
income_list = []  
for face, coupon in pairs:  
    income = face * coupon / 100  
    income_list.append(income)
```

As a list comprehension

```
income_list = [  
    face * coupon / 100  
    for face, coupon in pairs  
]
```

[]

The brackets say: build a list

face * coupon / 100

The calculation for each item

for face, coupon in pairs

Where the items come from

Writing your own function

```
def current_yield(coupon, price):  
    """  
    Take the coupon (in percent) and the price  
    and return the current yield in percent.  
    """  
    return coupon / price * 100
```

def and a name

Starts the definition. The name is how you call it.

Arguments

The inputs it needs, in the brackets.

Docstring

Says in plain English what goes in and what comes out.

return

Hands the answer back.

print

Shows the value on screen. Then it is gone.

return

Hands the value back, so you can use it next.

Functions that take any number of inputs

```
def total_face(face1, face2, face3, face4):
```

Fixed

Works for exactly four positions.

```
def total_face(*positions):
```

One star

Accepts as many values as it is given.

```
def total_income(*incomes, months=12):
```

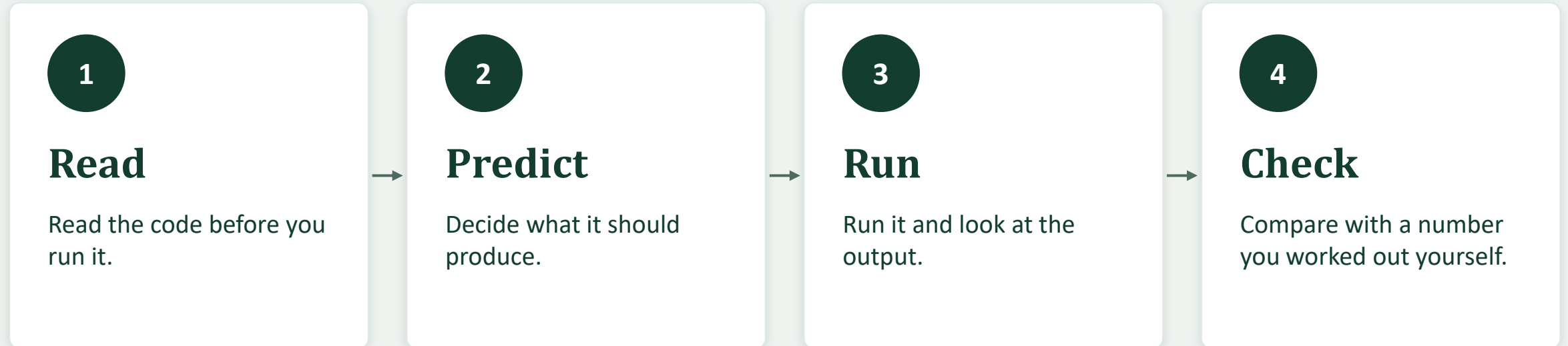
A default

Uses 12 months unless you say otherwise.

When you see a star, read it as: any number of these.

The AI check

Every notebook ends with code written in the style of an AI assistant. It has one bug. Find it.



Code that runs is not the same as code that is right.

What you can do now

- Store a bond in variables and calculate with it
- Hold a portfolio in lists and a dictionary, and total it
- Label a bond as premium or discount, investment grade or high yield
- Repeat a calculation across every position with a loop
- Write a function and use it again and again

Tomorrow Files and Folders: filing, opening, and reading the desk's daily holdings file